Ex.no:4b	
	EXPLORATORY DATA ANALYSIS (EDA) – DATA INSPECTION, FILTERING, STATISTICAL ANALYSIS AND CORRELATION

AIM

To perform data inspection, filtering, sorting, grouping, aggregation, descriptive statistical analysis, and correlation analysis on the Vehicle Analysis dataset using Pandas and visualize the analytical results using Matplotlib.

ALGORITHM

Algorithm 1: Dataset Inspection

1. Import the Vehicle Analysis dataset.
2. Display the first few records of the dataset.
3. Check the dimensions of the dataset.
4. Display the data types of the attributes.
5. Generate a statistical summary of the numerical attributes.

Algorithm 2: Filtering Data

1. Select the required attribute.
2. Specify a suitable filtering condition.
3. Apply the condition to the dataset.
4. Display the filtered records.

Algorithm 3: Sorting Data

1. Select the attribute to be sorted.
2. Apply the sort_values() function.
3. Specify the sorting order.
4. Display the sorted dataset.

Algorithm 4: Statistical Analysis

1. Select the required numerical attribute.
2. Calculate the mean.
3. Calculate the median.
4. Calculate the mode.
5. Calculate the standard deviation.
6. Display the calculated statistical measures.

Algorithm 5: Grouping and Aggregation

1. Select a categorical attribute such as region or country.
2. Apply the groupby() function.
3. Select the required numerical attribute.

4. Calculate an aggregate measure such as mean or sum.
5. Display the grouped results.

Algorithm 6: Correlation Analysis

1. Select the required numerical attributes.
2. Apply the corr() function.
3. Generate the correlation matrix.
4. Analyze the correlation coefficient between the selected attributes.
5. Visualize the relationship using an appropriate graph.

CODE

```
import kagglehub

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

import os

# ============================================================
# LOAD DATASET FROM KAGGLE
# ============================================================
path = kagglehub.dataset_download("nehalbirla/vehicle-dataset-from-cardekho")
print("Dataset Path:")
print(path)

# Find CSV file automatically

csv_files = [f for f in os.listdir(path) if f.endswith(".csv")]
print("\nCSV Files Found:")

print(csv_files)

csv_file = csv_files[0]

# Load Dataset

df = pd.read_csv(

    os.path.join(path, csv_file),

    encoding="latin1",
    low_memory=False
```

```
)

# Clean column names

df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")

print("\nDataset Loaded Successfully")

print("\nColumn Names:")

print(df.columns.tolist())

# ============================================================
# PROGRAM 1 - DATA INSPECTION
# ============================================================

print("\nFirst 5 Records")

print(df.head())

print("\nLast 5 Records")

print(df.tail())
# ============================================================
# PROGRAM 2 - SHAPE
# ============================================================

print("\nShape of Dataset")

print(df.shape)
print("\nRows =", df.shape[0])

print("Columns =", df.shape[1])

# ============================================================
# PROGRAM 3 - DATA TYPES
# ============================================================

print("\nData Types")
print(df.dtypes)
print("\nDataset Info")
df.info()

# ============================================================
# PROGRAM 4 - FILTERING
# Selling Price > 10 Lakhs
# ============================================================
```

```
high_price = df[df["selling_price"] > 1000000]
print("\nVehicles with Selling Price Greater Than 10 Lakhs")

print(

   high_price[

      [

         "car_name",

         "year",

         "selling_price",

         "present_price",

         "fuel_type"

      ]

   ].head(10)

)

# ============================================================
# PROGRAM 5 - FILTER VEHICLES WITH AUTOMATIC TRANSMISSION
# ============================================================

automatic = df[df["transmission"].str.lower() == "automatic"]


print("\nVehicles with Automatic Transmission")


print(

   automatic[

      [

         "car_name",

         "year",

         "selling_price",

         "fuel_type",
```

```
            "transmission"
        ]
    ].head(10)
)
# ============================================================
# PROGRAM 6 - SORTING
# Sort by Selling Price
# ============================================================
sorted_df = df.sort_values(
    by="selling_price",
    ascending=False
print("\nTop Vehicles by Selling Price")
print(
    sorted_df[
        [
            "car_name",
            "year",
            "selling_price",
            "present_price",
            "kms_driven"
        ]
    ].head(10)
)
# ============================================================
# BAR CHART
# TOP 10 VEHICLES BY SELLING PRICE
# ============================================================
```

```
top10 = sorted_df.head(10)

plt.figure(figsize=(10, 5))

plt.bar(
    range(len(top10)),
    top10["selling_price"]
)

plt.xticks(
    range(len(top10)),
    top10["car_name"],
    rotation=90
)

plt.xlabel("Car Name")
plt.ylabel("Selling Price")
plt.title("Top 10 Vehicles by Selling Price")
plt.show()

# ============================================================
# PROGRAM 7 - STATISTICAL ANALYSIS
# ============================================================

print("\nMean Selling Price =")
print(df["selling_price"].mean())
print("\nMedian Selling Price =")
print(df["selling_price"].median())
print("\nMode Selling Price =")
```

```
print(df["selling_price"].mode())

print("\nStandard Deviation =")

print(df["selling_price"].std())

print("\nMinimum Selling Price =")

print(df["selling_price"].min())

print("\nMaximum Selling Price =")

print(df["selling_price"].max())


# ============================================================
# PROGRAM 8 - DESCRIPTIVE STATISTICS
# ============================================================

print("\nDescriptive Statistics")


print(
   df[
      [
         "year",
         "selling_price",
         "present_price",
         "kms_driven"
      ]
   ].describe()


# ============================================================
# PROGRAM 9 - GROUPING
# Average Selling Price by Fuel Type
# ============================================================

group = df.groupby(
```

```
    "fuel_type"
)["selling_price"].mean().sort_values(ascending=False)

print("\nAverage Selling Price by Fuel Type")
print(group)
# Visualization
group.plot(
    kind="bar",
    figsize=(10, 5)
)
plt.xlabel("Fuel Type")
plt.ylabel("Average Selling Price")
plt.title("Average Selling Price by Fuel Type")
plt.xticks(rotation=0)
plt.show()

# ============================================================
# PROGRAM 10 - AGGREGATION
# ============================================================
summary = df.groupby("fuel_type").agg({
    "selling_price": "mean",
    "present  price": "mean",
    "kms_driven": "mean"
})

print("\nFuel Type Aggregation")
```

```
print(summary)

# ============================================================
# PROGRAM 11 - CORRELATION
# ============================================================

numeric = df.select_dtypes(

    include=np.number

)


corr = numeric.corr()


print("\nCorrelation Matrix")


print(corr)


plt.figure(figsize=(10, 7))


sns.heatmap(

    corr,

    annot=True,

    cmap="coolwarm"

)


plt.title("Vehicle Dataset Correlation Heatmap")


plt.show()


# ============================================================
# PROGRAM 12 - SCATTER PLOT
# Year vs Selling Price
# ============================================================
```

```
plt.figure(figsize=(7, 5))

plt.scatter(
    df["year"],
    df["selling_price"]
)

plt.xlabel("Year")
plt.ylabel("Selling Price")
plt.title("Year vs Selling Price")
plt.show()

# ============================================================
# PROGRAM 13 - HISTOGRAM
# Selling Price Distribution
# ============================================================

plt.figure(figsize=(7, 5))

plt.hist(
    df["selling_price"].dropna(),
    bins=30
)

plt.xlabel("Selling Price")
plt.ylabel("Frequency")
plt.title("Distribution of Vehicle Selling Prices")
plt.show()
```

```
# ============================================================
# PROGRAM 14 - BOXPLOT
# ============================================================

plt.figure(figsize=(7, 5))


sns.boxplot(

    x=df["selling_price"].dropna()


plt.title("Selling Price Boxplot")

plt.xlabel("Selling Price")
plt.show()
# ============================================================
# PROGRAM 15 - VALUE COUNTS
# ============================================================

print("\nFuel Type Counts")


print(

    df["fuel_type"].value_counts()

)

# ============================================================
# PROGRAM 16 - UNIQUE VALUES
# ============================================================

print("\nUnique Car Names")

print(

    df["car_name"].unique()

)

print("\nUnique Fuel Types")


print(

    df["fuel  type"].unique()
```

```
)
print("\nUnique Seller Types")

print(

    df["seller_type"].unique()

)

print("\nUnique Transmissions"

print(

    df["transmission"].unique()

)

# ============================================================
# PROGRAM 17 - TOP 10 CAR NAMES
# ============================================================

print("\nTop 10 Car Names by Number of Vehicles")

car_counts = (

    df["car_name"]

    .value_counts()

    .head(10)

)

print(car_counts)

# ============================================================
# PROGRAM 18 - VEHICLES BY YEAR
# ============================================================

print("\nNumber of Vehicles by Year")

year_counts = (

    df["year"]

    .value_counts()
    .sort  index()
)
```

```
print(year_counts)


# ============================================================
# PROGRAM 19 - TOP 10 CAR NAMES BY SELLING PRICE
# ============================================================

print("\nTop 10 Car Names by Total Selling Price")

car_price = (

    df.groupby("car_name")["selling_price"]

    .sum()

    .sort_values(ascending=False)

    .head(10)

)
print(car_price)


# ============================================================
# PROGRAM 20 - MOST COMMON FUEL TYPES
# ============================================================

print("\nMost Common Fuel Types")

fuel_types = (

    df["fuel_type"]

    .value_counts()

    .head(10)

)

print(fuel  types)

# ============================================================
# PROGRAM 21 - EXPORT DATA
# ============================================================

df.to_csv(

    "vehicle_data.csv",
```

```
    index=False
)
df.to_excel(
    "vehicle_data.xlsx",
    index=False
)
df.to_json(
    "vehicle_data.json",
    orient="records",
    indent=4
)
print("\nCSV, Excel and JSON files exported successfully.")


# ============================================================
# FINAL REPORT
# ============================================================

print("\n================================================")

print("EXPERIMENT COMPLETED SUCCESSFULLY")

print("================================================")

print("Dataset Shape :", df.shape)

print(
    "Missing Values :",
    df.isnull().sum().sum()
)
print(
    "Total Vehicles :",
    len(df)
```

```
print(
    "Total Selling Price :",
    df["selling_price"].sum()
)
print(
    "Average Selling Price :",
    df["selling_price"].mean()
)
print(
    "Number of Car Names :",
    df["car_name"].nunique()
)
print(
    "Number of Fuel Types :",
    df["fuel_type"].nunique()
)
print(
    "Number of Seller Types :",
    df["seller_type"].nunique()
)
print(
    "Number of Transmissions :",
    df["transmission"].nunique()
)
```

```
print("Correlation Matrix Generated")

print("================================================")
```

OUTPUT:

```
 Dataset Path:
/kaggle/input/vehicle-dataset-from-cardekho

CSV Files Found:
['car data.csv', 'car details v4.csv', 'CAR DETAILS FROM CAR DEKHO.csv', 'Car details v3.csv']

Dataset Loaded Successfully

Column Names:
['car_name', 'year', 'selling_price', 'present_price', 'kms_driven', 'fuel_type', 'seller_type', 'transmission', 'owner']

First 5 Records
  car_name  year  selling_price  present_price  kms_driven fuel_type  \
0     ritz  2014           3.35           5.59       27000    Petrol
1      sx4  2013           4.75           9.54       43000    Diesel
2     ciaz  2017           7.25           9.85        6900    Petrol
3  wagon r  2011           2.85           4.15        5200    Petrol
4    swift  2014           4.60           6.87       42450    Diesel

  seller_type transmission  owner
0      Dealer       Manual      0
1      Dealer       Manual      0
2      Dealer       Manual      0
3      Dealer       Manual      0
4      Dealer       Manual      0

Last 5 Records
    car_name  year  selling_price  present_price  kms_driven fuel_type  \
296     city  2016           9.50           11.6       33988    Diesel
297     brio  2015           4.00            5.9       60000    Petrol
298     city  2009           3.35           11.0       87934    Petrol
299     city  2017          11.50           12.5        9000    Diesel
300     brio  2016           5.30            5.9        5464    Petrol

    seller_type transmission  owner
296      Dealer       Manual      0
297      Dealer       Manual      0
298      Dealer       Manual      0
299      Dealer       Manual      0
300      Dealer       Manual      0

Shape of Dataset
(301, 9)

Rows = 301
Columns = 9
```

Data Types
```
car_name          object
year               int64
selling_price    float64
present_price    float64
kms_driven         int64
fuel_type         object
seller_type       object
transmission      object
owner              int64
dtype: object
```

Dataset Info
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 301 entries, 0 to 300
Data columns (total 9 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   car_name       301 non-null    object
 1   year           301 non-null    int64
 2   selling_price  301 non-null    float64
 3   present_price  301 non-null    float64
 4   kms_driven     301 non-null    int64
 5   fuel_type      301 non-null    object
 6   seller_type    301 non-null    object
 7   transmission   301 non-null    object
 8   owner          301 non-null    int64
dtypes: float64(2), int64(3), object(4)
memory usage: 21.3+ KB
```

Vehicles with Selling Price Greater Than 10 Lakhs
```
Empty DataFrame
Columns: [car_name, year, selling_price, present_price, fuel_type]
Index: []
```

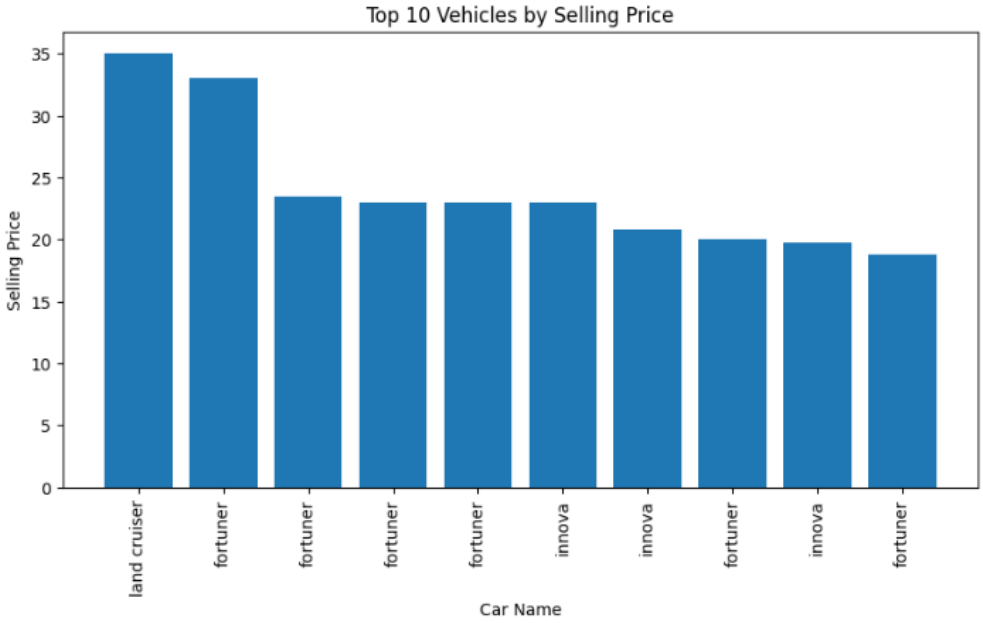
Vehicles with Automatic Transmission

	car_name	year	selling_price	fuel_type	transmission
12	ciaz	2015	7.50	Petrol	Automatic
22	sx4	2011	4.40	Petrol	Automatic
40	baleno	2016	5.85	Petrol	Automatic
45	ciaz	2014	7.50	Petrol	Automatic
49	ciaz	2017	7.75	Petrol	Automatic
50	fortuner	2012	14.90	Diesel	Automatic
51	fortuner	2015	23.00	Diesel	Automatic
52	innova	2017	18.00	Diesel	Automatic
53	fortuner	2013	16.00	Diesel	Automatic
55	corolla altis	2009	3.60	Petrol	Automatic

Top Vehicles by Selling Price

	car_name	year	selling_price	present_price	kms_driven
86	land cruiser	2010	35.00	92.60	78000
64	fortuner	2017	33.00	36.23	6000
63	fortuner	2015	23.50	35.96	47000
51	fortuner	2015	23.00	30.61	40000
93	fortuner	2015	23.00	30.61	40000
82	innova	2017	23.00	25.39	15000
96	innova	2016	20.75	25.39	29000

59	fortuner	2014	19.99	35.96	41000
66	innova	2017	19.75	23.15	11000
62	fortuner	2014	18.75	35.96	78000



Mean Selling Price =
4.661295681063123

Median Selling Price =
3.6

Mode Selling Price =
0 0.45
1 0.60
Name: selling_price, dtype: float64

Standard Deviation =
5.082811556177804

Minimum Selling Price =
0.1

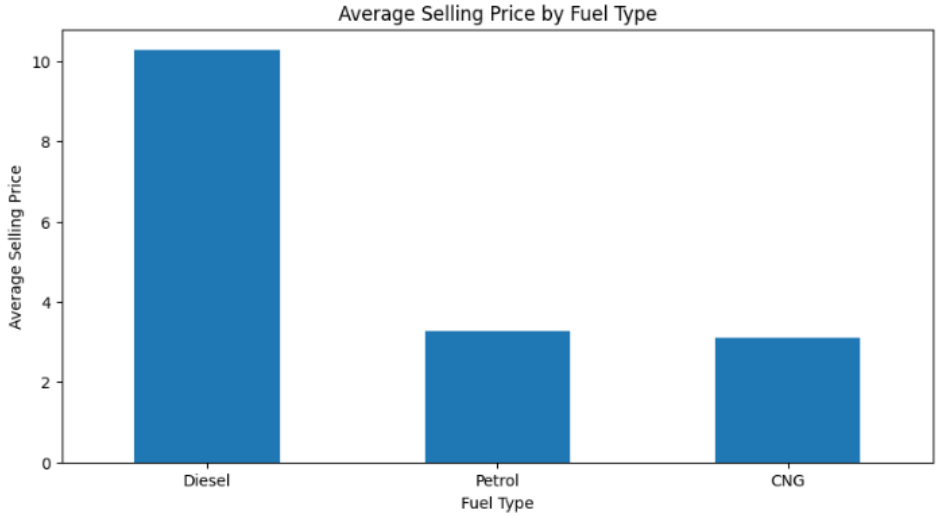
Maximum Selling Price =
35.0

Descriptive Statistics

	year	selling_price	present_price	kms_driven
count	301.000000	301.000000	301.000000	301.000000
mean	2013.627907	4.661296	7.628472	36947.205980
std	2.891554	5.082812	8.644115	38886.883882
min	2003.000000	0.100000	0.320000	500.000000
25%	2012.000000	0.900000	1.200000	15000.000000
50%	2014.000000	3.600000	6.400000	32000.000000
75%	2016.000000	6.000000	9.900000	48767.000000
max	2018.000000	35.000000	92.600000	500000.000000

Average Selling Price by Fuel Type
fuel_type

Diesel 10.278500
Petrol 3.264184
CNG 3.100000
Name: selling_price, dtype: float64

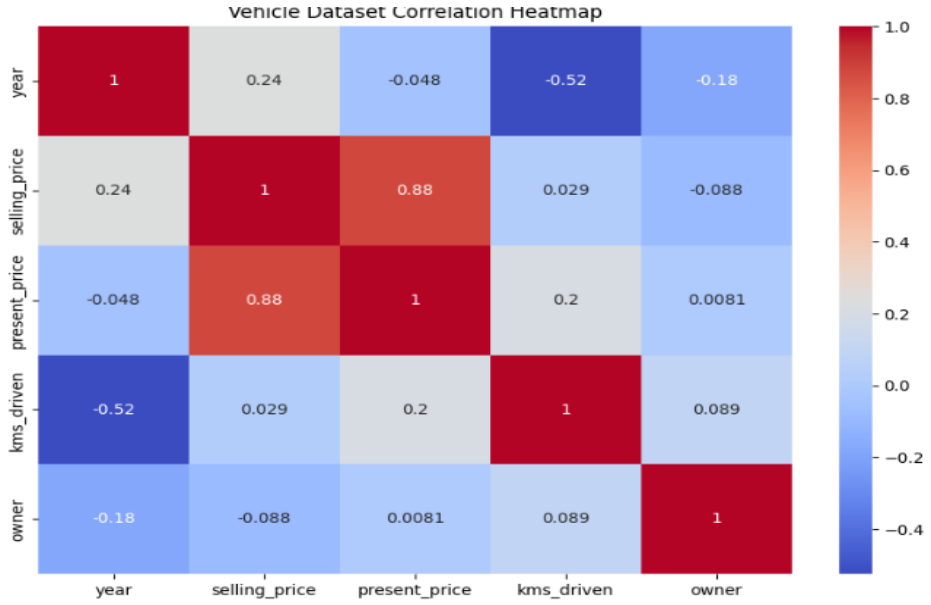


Fuel Type Aggregation

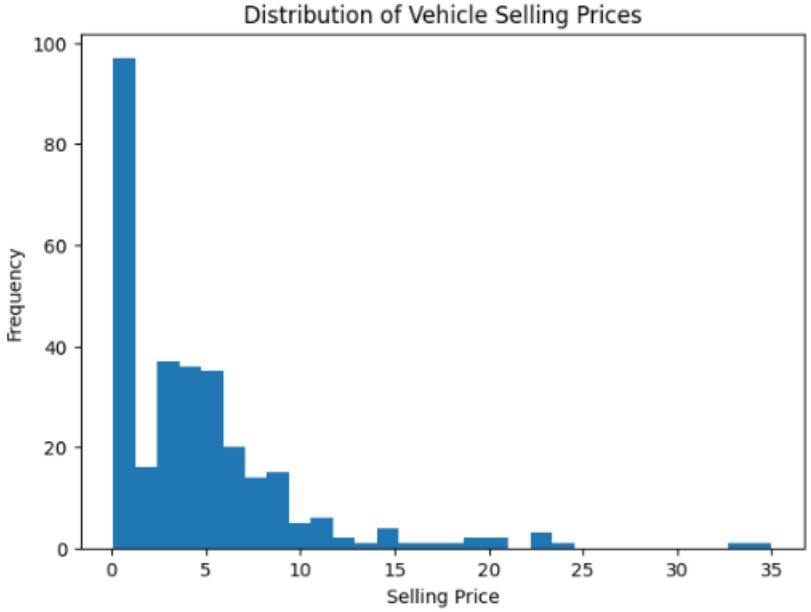
fuel_type	selling_price	present_price	kms_driven
CNG	3.100000	6.415000	42749.000000
Diesel	10.278500	15.814500	50369.916667
Petrol	3.264184	5.583556	33528.937238

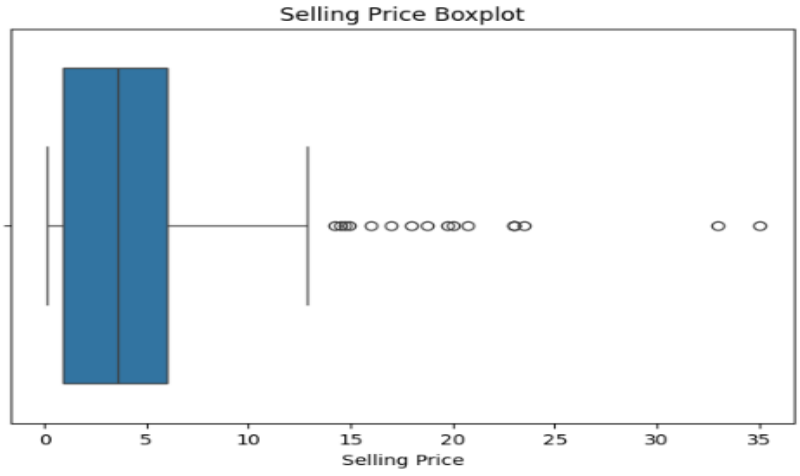
Correlation Matrix

	year	selling_price	present_price	kms_driven	owner
year	1.000000	0.236141	-0.047584	-0.524342	-0.182104
selling_price	0.236141	1.000000	0.878983	0.029187	-0.088344
present_price	-0.047584	0.878983	1.000000	0.203647	0.008057
kms_driven	-0.524342	0.029187	0.203647	1.000000	0.089216
owner	-0.182104	-0.088344	0.008057	0.089216	1.000000




Year vs Selling Price
Selling Price
Year
0
5
10
15
20
25
30
35
2004
2006
2008
2010
2012
2014
2016
2018


Distribution of Vehicle Selling Prices
Frequency
Selling Price
0
20
40
60
80
100
0
5
10
15
20
25
30
35


Selling Price Boxplot
Selling Price
0
5
10
15
20
25
30
35

Fuel Type Counts

fuel_type

Petrol 239

Diesel 60

CNG 2

Name: count, dtype: int64

Unique Car Names

['ritz' 'sx4' 'ciaz' 'wagon r' 'swift' 'vitara brezza' 's cross'

'alto 800' 'ertiga' 'dzire' 'alto k10' 'ignis' '800' 'baleno' 'omni'

'fortuner' 'innova' 'corolla altis' 'etios cross' 'etios g' 'etios liva'

'corolla' 'etios gd' 'camry' 'land cruiser' 'Royal Enfield Thunder 500'

'UM Renegade Mojave' 'KTM RC200' 'Bajaj Dominar 400'

'Royal Enfield Classic 350' 'KTM RC390' 'Hyosung GT250R'

'Royal Enfield Thunder 350' 'KTM 390 Duke ' 'Mahindra Mojo XT300'

'Bajaj Pulsar RS200' 'Royal Enfield Bullet 350'

'Royal Enfield Classic 500' 'Bajaj Avenger 220' 'Bajaj Avenger 150'

'Honda CB Hornet 160R' 'Yamaha FZ S V 2.0' 'Yamaha FZ 16'

'TVS Apache RTR 160' 'Bajaj Pulsar 150' 'Honda CBR 150' 'Hero Extreme'

'Bajaj Avenger 220 dtsi' 'Bajaj Avenger 150 street' 'Yamaha FZ v 2.0'

'Bajaj Pulsar NS 200' 'Bajaj Pulsar 220 F' 'TVS Apache RTR 180'

'Hero Passion X pro' 'Bajaj Pulsar NS 200' 'Yamaha Fazer '

'Honda Activa 4G' 'TVS Sport ' 'Honda Dream Yuga '

'Bajaj Avenger Street 220' 'Hero Splender iSmart' 'Activa 3g'

'Hero Passion Pro' 'Honda CB Trigger' 'Yamaha FZ S '

'Bajaj Pulsar 135 LS' 'Activa 4g' 'Honda CB Unicorn'

'Hero Honda CBZ extreme' 'Honda Karizma' 'Honda Activa 125' 'TVS Jupyter'

'Hero Honda Passion Pro' 'Hero Splender Plus' 'Honda CB Shine'

'Bajaj Discover 100' 'Suzuki Access 125' 'TVS Wego' 'Honda CB twister'

'Hero Glamour' 'Hero Super Splendor' 'Bajaj Discover 125' 'Hero Hunk'

'Hero Ignitor Disc' 'Hero CBZ Xtreme' 'Bajaj ct 100' 'i20' 'grand i10'

'i10' 'eon' 'xcent' 'elantra' 'creta' 'verna' 'city' 'brio' 'amaze'

'jazz'

Unique Fuel Types

['Petrol' 'Diesel' 'CNG'

Unique Seller Types

['Dealer' 'Individual']

Unique Transmissions

['Manual' 'Automatic']

Top 10 Car Names by Number of Vehicles

car_name

city 26

corolla altis 16

verna 14

fortuner 11

brio 10

ciaz 9

i20 9

innova 9

grand i10 8

Royal Enfield Classic 350 7

Name: count, dtype: int64

Number of Vehicles by Year

year

2003 2

2004 1

2005 4

2006 4

2007 2

2008 7

2009 6

2010 15

2011 19

2012 23

2013 33

2014 38

2015 61

2016 50

2017 35

2018 1

Name: count, dtype: int64

Top 10 Car Names by Total Selling Price

car_name

fortuner 205.54

city 192.90

innova 115.00

corolla altis 114.93

verna 85.51

ciaz 67.25

brio 47.45

i20 42.90

jazz 40.80

ertiga 40.65

Name: selling_price, dtype: float64

Most Common Fuel Types

fuel_type

Petrol 239

Diesel 60

CNG 2

Name: count, dtype: int64

CSV, Excel and JSON files exported successfully

==========

EXPERIMENT COMPLETED SUCCESSFULLY

==========

Dataset Shape : (301, 9)

Missing Values : 0

Total Vehicles : 301

Total Selling Price : 1403.0500000000002

Average Selling Price : 4.661295681063123

Number of Car Names : 98

Number of Fuel Types : 3

Number of Seller Types : 2

Number of Transmissions : 2

Correlation Matrix Generated

RESULT